1. Why Do We Need Historical Recovery?

Imagine a production table containing millions of orders.

A pipeline accidentally:

- Updates the wrong records
- Deletes valid records
- Writes incorrect data

Instead of rebuilding everything from the source, Delta Lake provides features such as:

DESCRIBE HISTORY
Time Travel
RESTORE
VACUUM

These allow us to inspect historical states, recover data, and manage old files.

However, historical recovery works only while the required historical transaction information and data files are still available. Class 08 — Delta Lake Time Trav…

---

2. Delta Table Versions

Every successful data-changing commit creates a new Delta table version.

Example flow:

Version 0 → Table Created
Version 1 → INSERT
Version 2 → UPDATE
Version 3 → DELETE

Delta maintains a record of these committed operations.

This allows us to understand:

- What changed
- When it changed
- Which operation happened
- Which table version represents that state

---

3. Inspecting Table History

Use:

DESCRIBE HISTORY datax_db.orders_time_travel_demo;

This can show information such as:

- Version
- Timestamp
- Operation
- Operation metrics

For a fresh example, history may look like:

Version 0 → CREATE TABLE AS SELECT
Version 1 → INSERT
Version 2 → UPDATE
Version 3 → DELETE

Exact version numbers can differ if the table has already been modified. Class 08 — Delta Lake Time Trav…

---

4. Why Versions Are Important

Suppose we insert:

Order 1011
Status = PENDING

Then update:

PENDING → COMPLETED

Then delete the order.

The table goes through multiple states:

Version 1

1011 → PENDING

Version 2
1011 → COMPLETED

Version 3
1011 → Deleted

Delta keeps track of these committed states.

---

5. What Is Delta Time Travel?

Time Travel allows us to query an older version of a Delta table.

It does not modify the current table.

It only lets us inspect how the table looked earlier.

You can Time Travel using:

- Version number
- Timestamp

---

6. Version-Based Time Travel

Suppose version 1 represents the table after inserting order `1011`.

```
SELECT *
FROM datax_db.orders_time_travel_demo
VERSION AS OF 1
ORDER BY order_id;
```

At this version:

1011 → PENDING

Now query version 2:

```
SELECT *
```

```
FROM datax_db.orders_time_travel_demo
VERSION AS OF 2
WHERE order_id = 1011;
```

Result:

1011 → COMPLETED

Version 3:

```
SELECT *
FROM datax_db.orders_time_travel_demo
VERSION AS OF 3
WHERE order_id = 1011;
```

No record is returned because the order had been deleted by that version.

Key Point

Time Travel
→ Reads historical data
→ Does NOT modify the current table

Class 08 — Delta Lake Time Trav…

---

7. Timestamp-Based Time Travel

Sometimes we may not know the exact version number.

But we may know:

> The issue happened around 2 PM.

First check:

```
DESCRIBE HISTORY datax_db.orders_time_travel_demo;
```

Find the required timestamp.

Then query:

```
SELECT *
FROM datax_db.orders_time_travel_demo
TIMESTAMP AS OF '2026-10-11 14:00:00';
```

Use an actual timestamp from your own table history.

Important

Timestamp-based Time Travel also requires the necessary historical files to still exist.

---

8. Time Travel vs RESTORE

This distinction is extremely important.

Time Travel

Reads an older version
Does not modify current table
Useful for investigation

RESTORE

Changes current table
Recovers an earlier state
Creates a new Delta commit

Easy Way to Remember

Time Travel → LOOK BACK

RESTORE → GO BACK
by creating a new current state

Class 08 — Delta Lake Time Trav…

---

9. Recovering Data Using RESTORE

Suppose the DELETE was accidental.

Version 2 contained the correct data.

Restore:

```
RESTORE TABLE datax_db.orders_time_travel_demo
TO VERSION AS OF 2;
```

Now check:

```
SELECT *
FROM datax_db.orders_time_travel_demo
WHERE order_id = 1011;
```

Expected:

```
order_id     → 1011
order_status → COMPLETED
```

The order has been recovered.

---

10. RESTORE Does Not Remove History

This is important.

Suppose history was:

```
Version 0 → CREATE
Version 1 → INSERT
Version 2 → UPDATE
Version 3 → DELETE
```

After:

```
RESTORE ... TO VERSION AS OF 2
```

Delta does not reset the table back to version 2.

Instead, it creates another commit:

```
Version 4 → RESTORE
```

So history may become:

```
0 → CREATE
1 → INSERT
2 → UPDATE
3 → DELETE
```

4 → RESTORE

This means even the recovery action itself is tracked. Class 08 — Delta Lake Time Trav…

---

11. Why RESTORE Should Be Used Carefully

Before restoring:

1. Inspect `DESCRIBE HISTORY`.
2. Use Time Travel to inspect the version.
3. Verify that the data is correct.
4. Only then perform `RESTORE`.

Why?

Because restoring an older state can remove legitimate changes made after that version from the current table state.

---

12. What Happens to Old Data Files?

After an `UPDATE` or `DELETE`, older files may no longer belong to the current table snapshot.

However, Delta may keep those files for a retention period.

Why?

Because historical versions may still depend on them.

Conceptually:

Current Version
↓
Active Data Files

Older Versions
↓

May depend on older files

Keeping these files enables historical recovery.

But keeping obsolete files forever would increase storage usage.

That is why Delta provides `VACUUM`.

---

13. What Is VACUUM?

`VACUUM` removes eligible old data files that:

- Are no longer required by the current Delta table state
- Are older than the configured retention threshold

Example:

A table has gone through hundreds of:

UPDATE
DELETE
WRITE

Some old files may no longer be required by the latest table state.

`VACUUM` can remove eligible files.

Important

`VACUUM` is **not** used to delete records from the current table.

It is a storage cleanup operation. Class 08 — Delta Lake Time Trav…

---

14. VACUUM Retention

The class uses:

168 hours

which equals:

7 days

The default VACUUM retention threshold is generally 7 days, though retention settings and runtime behavior can vary.

Aggressively reducing retention can affect:

- Historical queries
- RESTORE
- Time Travel
- Concurrent workloads

---

15. VACUUM DRY RUN

Before deleting anything, you can inspect eligible files.

```
VACUUM datax_db.orders_time_travel_demo
RETAIN 168 HOURS DRY RUN;
```

DRY RUN

Shows which files are eligible for cleanup.

It does **not** delete them.

For a newly created table, you may see no eligible files.

That is completely normal.

---

16. Running VACUUM

Actual cleanup:

```
VACUUM datax_db.orders_time_travel_demo
RETAIN 168 HOURS;
```

This removes files that meet the cleanup criteria.

Do not reduce retention only to force files to disappear during a demonstration.

---

17. Time Travel and VACUUM Are Connected

Time Travel needs:

Historical Transaction Information
+
Historical Data Files

If `VACUUM` removes files required by an older version:

Older Version
↓
Required Files Removed
↓
Time Travel may fail

So:

More retention
→ More historical recovery possibility
→ More storage usage

Aggressive cleanup
→ Less storage
→ Smaller historical recovery window

Class 08 — Delta Lake Time Trav…

---

18. History Does Not Guarantee Time Travel

Suppose:

DESCRIBE HISTORY table_name;

still shows version 2.

Does that guarantee this works?

SELECT *
FROM table_name
VERSION AS OF 2;

No.

Why?

Because `DESCRIBE HISTORY` may still contain information that the transaction occurred, while the data files required to reconstruct that version may already be unavailable.

Remember

History
→ Transaction existed

Time Travel
→ Transaction information + required historical data must exist

---

19. Time Travel Is Not a Backup Strategy

Time Travel is useful for:

- Investigation
- Comparing historical states
- Recovering recent mistakes

But it should not be treated as a complete backup and disaster-recovery solution.

Historical availability depends on retention.

---

20. Production Recovery Flow

A safer production approach is:

Issue Detected
↓
DESCRIBE HISTORY
↓
Identify Candidate Version
↓
Time Travel
↓
Validate Historical Data
↓
RESTORE
↓
Validate Current Table

Do not immediately execute `RESTORE`.

---

21. RESTORE and Downstream Pipelines

RESTORE creates a new data-changing commit.

This matters when downstream pipelines use:

- Incremental processing
- Change-based processing

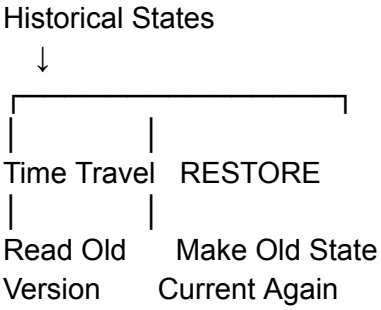
A restore may cause restored data to be processed again.

So recovery operations should be handled like any other production data change.

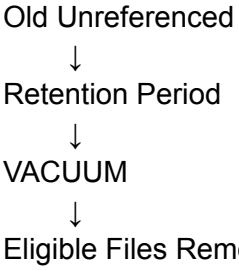
---

22. Complete Conceptual Flow

Delta Table
↓
Transactions
↓
Versions
↓
DESCRIBE HISTORY
↓



And eventually:



---

23. Time Travel vs RESTORE vs VACUUM

Feature	Purpose	Changes Current Table?
`DESCRIBE HISTORY`	Inspect table commits	No
Time Travel	Read historical state	No
`RESTORE`	Recover earlier state	Yes
`VACUUM`	Clean eligible old files	Does not delete current records

---

24. Commands to Remember

History

DESCRIBE HISTORY datax_db.orders_time_travel_demo;

Version Time Travel

```
SELECT *
FROM datax_db.orders_time_travel_demo
VERSION AS OF 2;
```

Timestamp Time Travel

```
SELECT *
FROM datax_db.orders_time_travel_demo
TIMESTAMP AS OF '<timestamp>';
```

RESTORE

```
RESTORE TABLE datax_db.orders_time_travel_demo
TO VERSION AS OF 2;
```

VACUUM Dry Run

```
VACUUM datax_db.orders_time_travel_demo
RETAIN 168 HOURS DRY RUN;
```

VACUUM

```
VACUUM datax_db.orders_time_travel_demo
RETAIN 168 HOURS;
```

---

25. Common Mistakes

Mistake 1: Thinking Time Travel Changes the Table

It does not.

Time Travel only reads a historical state.

Mistake 2: Thinking RESTORE Resets Version Numbers

It does not.

RESTORE creates a new commit.

Mistake 3: Restoring Without Checking History

Always identify and validate the intended version first.

Mistake 4: Assuming Every Version in History Is Still Queryable

Historical files may already have been removed.

Mistake 5: Using VACUUM Aggressively

Removing historical files can limit Time Travel and recovery.

Mistake 6: Treating VACUUM as DELETE

`VACUUM` cleans obsolete physical files.

It is not used to remove rows from the current table.

---

26. Quick Revision

DESCRIBE HISTORY
→ What happened?

Time Travel
→ What did the table look like?

RESTORE
→ Make an earlier state current again

VACUUM
→ Clean eligible old physical files

Retention
→ Controls how much historical recovery remains possible

---

27. Student Practice Assignment

Use:

datax_db.orders_time_travel_demo

Task 1 — Create a New Version

Insert order:

order_id → 1012
customer_id → C109
product_id → P103
order_date → 2026-09-07
quantity → 2
unit_price → 1499.00
order_status → PENDING

Task 2 — UPDATE

Change:

PENDING → COMPLETED

for order `1012`.

Verify the result.

Task 3 — DELETE

Delete:

order_id = 1012

Verify that it no longer appears in the current table.

Task 4 — Explore History

DESCRIBE HISTORY datax_db.orders_time_travel_demo;

Identify:

- Version before DELETE
- DELETE version

Task 5 — Time Travel

Query the version immediately before DELETE.

Confirm that:

order_id = 1012

still exists there.

Task 6 — RESTORE

Restore the table to the version immediately before DELETE.

Verify that order `1012` is recovered.

Task 7 — VACUUM

VACUUM datax_db.orders_time_travel_demo
RETAIN 168 HOURS DRY RUN;

Observe whether any files are eligible for cleanup.

For a newly created table, there may be none. Class 08 — Delta Lake Time Trav…

---

28. Interview-Relevant Questions

1. What is Delta Time Travel?

Time Travel allows us to query previous versions of a Delta table using a version number or timestamp, provided the required historical data is still available.

---

2. What is the difference between Time Travel and RESTORE?

Time Travel reads historical data without changing the current table.

RESTORE changes the current table so that it matches an earlier state and creates a new commit.

---

3. Does `DESCRIBE HISTORY` guarantee that an old version is readable?

No.

History metadata may remain even when required historical data files are no longer available.

---

4. What is VACUUM?

VACUUM removes eligible old data files that are no longer referenced by the current Delta table and meet the retention criteria.

---

5. Does VACUUM delete records from the current table?

No.

It cleans eligible obsolete physical files.

---

6. How are Time Travel and VACUUM related?

Time Travel needs historical data files.

VACUUM can remove old files, potentially making older versions unavailable.

---

7. Does RESTORE remove existing history?

No.

RESTORE itself creates a new transaction and table version.

---

8. Why should you use Time Travel before RESTORE?

To verify that the historical version actually contains the desired data before changing the current table.

---

9. Why shouldn't VACUUM retention be reduced aggressively?

Because it can remove files needed for historical recovery and may interfere with concurrent workloads.

---

10. Can we normally use `VACUUM RETAIN 0 HOURS`?

Databricks generally uses retention safety checks to prevent dangerously short retention settings. Disabling those safeguards can put concurrent workloads and historical recovery at risk and is not recommended for normal production usage. Class 08 — Delta Lake Time Trav…

Key Takeaway

Every Delta Commit
↓
Creates a Version
↓
DESCRIBE HISTORY
↓
Time Travel
↓
Inspect Old Data
↓
RESTORE
↓
Recover if Required

Meanwhile...

Old Unreferenced Files
↓
Retention
↓
VACUUM
↓
Storage Cleanup

Delta Lake gives us powerful historical recovery capabilities, but those capabilities depend heavily on how we manage retention and old data files.